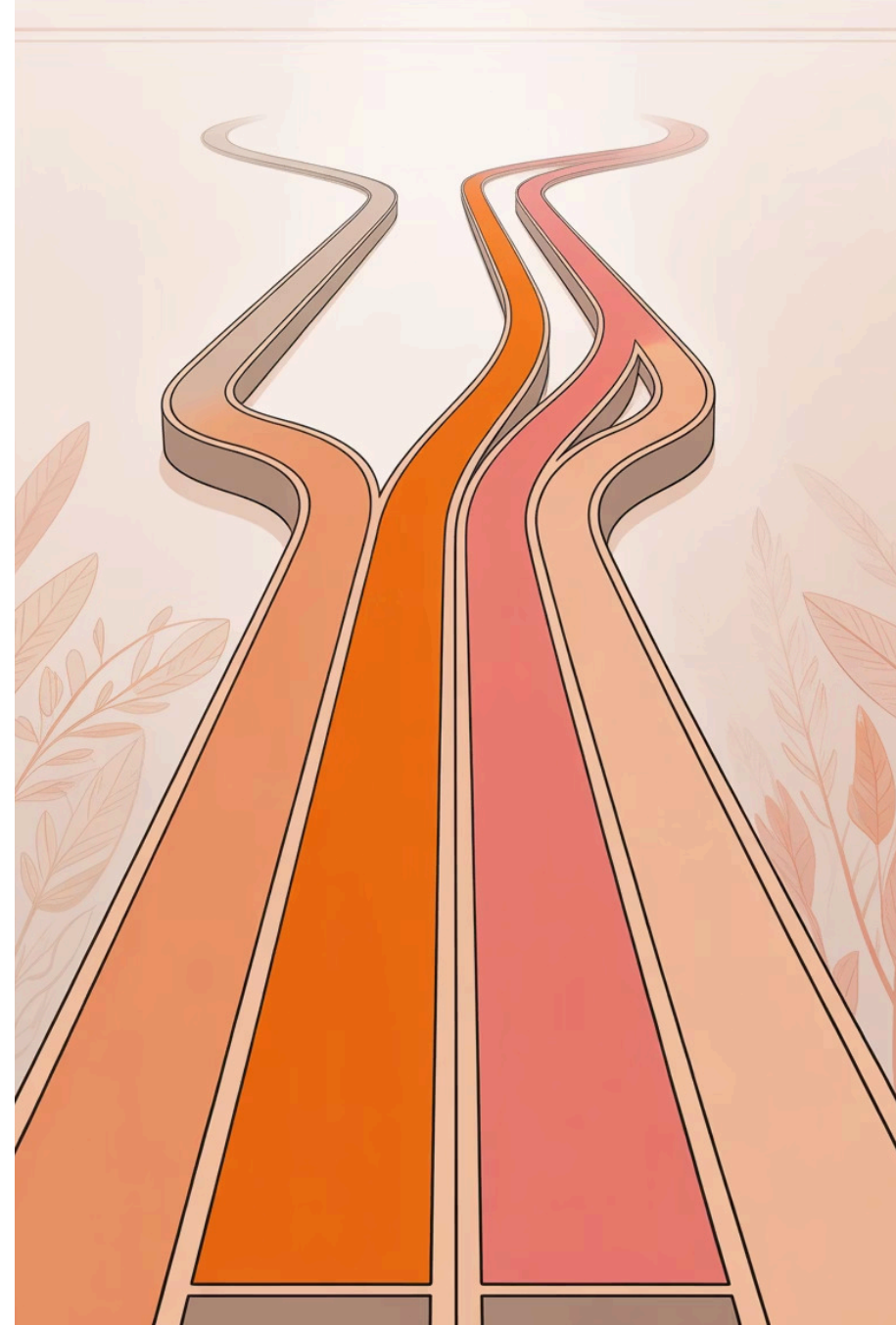


Paralelismo con Streams

Procesamiento secuencial vs procesamiento paralelo

En este módulo final de nuestra serie sobre Programación Funcional en Java, exploraremos cómo aprovechar el poder del procesamiento paralelo para mejorar el rendimiento de nuestras aplicaciones.



Procesamiento Secuencial: El Flujo Predeterminado

Por defecto, los Streams procesan los elementos **uno tras otro** en una única secuencia lineal:

```
List nombres = Arrays.asList(
    "Ana", "Carlos", "Elena", "David");

nombres.stream()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

Este procesamiento utiliza un solo hilo, independientemente del número de núcleos disponibles en el procesador.



En el procesamiento secuencial, cada elemento espera a que termine el procesamiento del elemento anterior antes de comenzar.

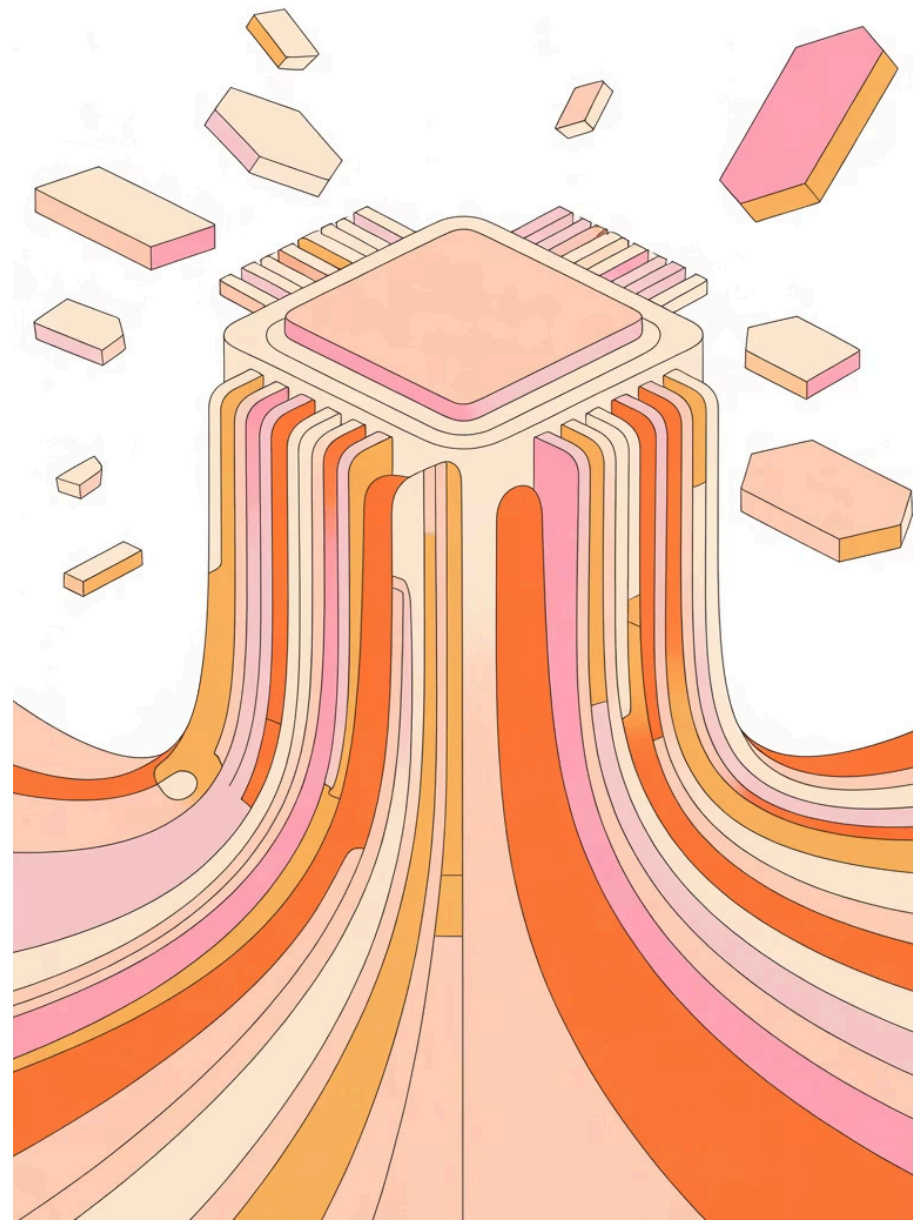
Procesamiento Paralelo: Dividir para Conquistar

Con `.parallelStream()` o `.parallel()` podemos dividir el trabajo entre **varios hilos** simultáneamente:

```
// Creando un stream paralelo desde una colección
nombres.parallelStream()
    .map(String::toUpperCase)
    .forEach(System.out::println);

// O convirtiendo un stream secuencial en paralelo
nombres.stream()
    .parallel()
    .map(String::toUpperCase)
    .forEach(System.out::println);
```

El runtime de Java divide automáticamente los datos entre los núcleos disponibles del procesador, procesa cada parte por separado y luego combina los resultados.



Beneficios del Procesamiento Paralelo



Mayor Velocidad

Reducción significativa del tiempo de procesamiento para colecciones grandes, especialmente en operaciones intensivas.



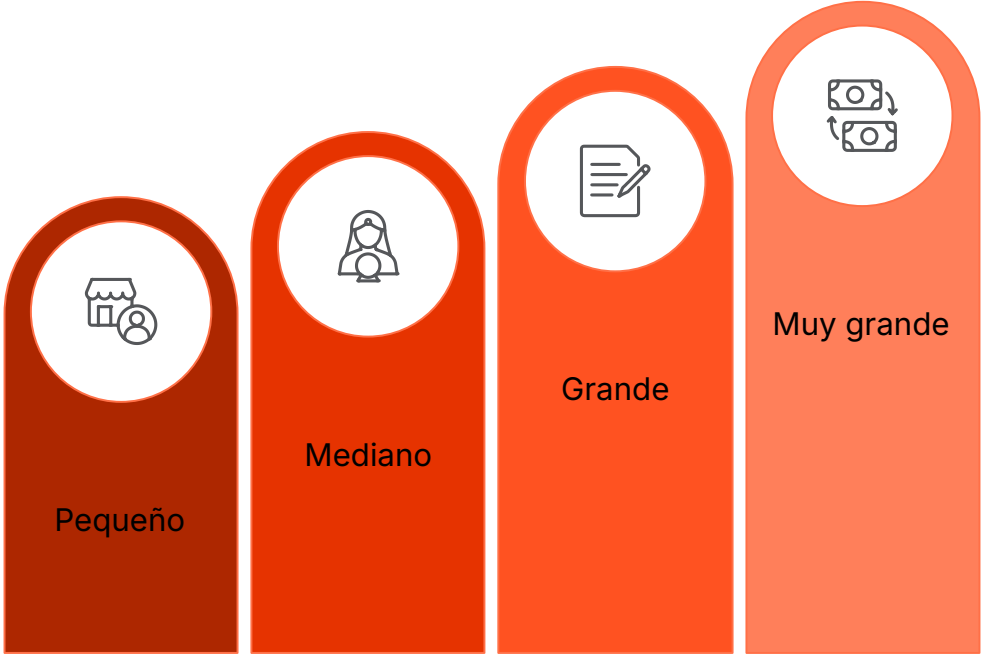
Escalabilidad Automática

Aprovecha automáticamente todos los núcleos disponibles sin configuración adicional.



Ideal para Tareas Independientes

Perfecto para operaciones donde cada elemento puede procesarse sin depender del resultado de otros.



A mayor cantidad de datos y complejidad de procesamiento, mayores son los beneficios del paralelismo.

Riesgos y Limitaciones

No Siempre Es Más Rápido

- Con **colecciones pequeñas**, el coste de dividir y combinar supera los beneficios
- Operaciones simples pueden ser más lentas en paralelo debido al overhead

Orden No Garantizado

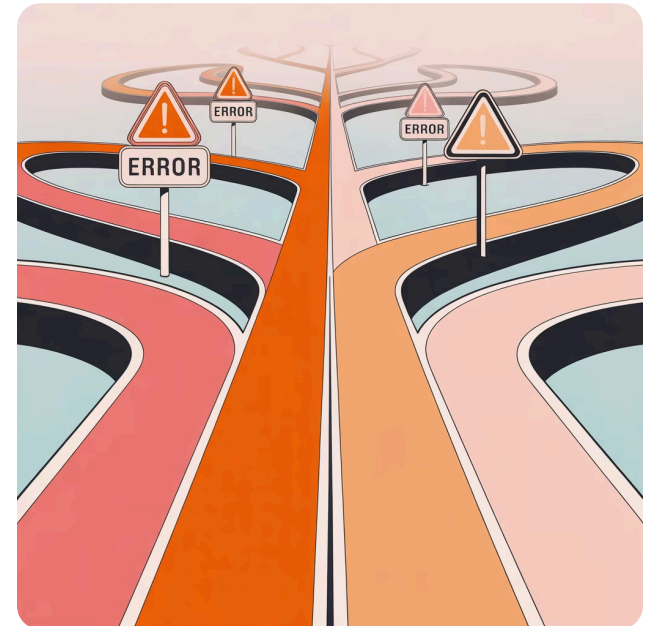
- El orden de procesamiento no se garantiza, a menos que uses `forEachOrdered()`
- Algunos colectores como `Collectors.toList()` no preservan el orden en paralelo

Problemas de Concurrencia

- Las operaciones deben ser **thread-safe** y preferiblemente sin estado
- Incrementar contadores o modificar variables compartidas puede causar resultados incorrectos

⚠ ¡Cuidado!

Los Streams paralelos utilizan el **ForkJoinPool común**. Si bloqueas hilos o realizas operaciones muy largas, podrías afectar a otras partes de tu aplicación que también lo utilicen.

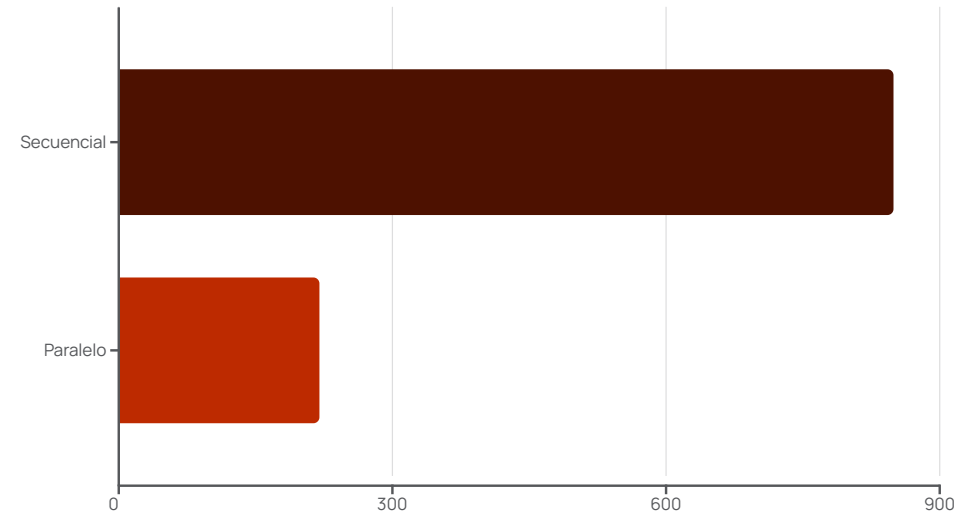


Ejemplo Comparativo: Secuencial vs Paralelo

```
List numeros = IntStream.rangeClosed(1, 10_000_000)
    .boxed()
    .collect(Collectors.toList());

// Prueba secuencial
long inicio = System.currentTimeMillis();
long sumaSecuencial = numeros.stream()
    .filter(n -> n % 2 == 0)
    .mapToLong(n -> n * n)
    .sum();
long tiempoSecuencial = System.currentTimeMillis() - inicio;

// Prueba paralela
inicio = System.currentTimeMillis();
long sumaParalela = numeros.parallelStream()
    .filter(n -> n % 2 == 0)
    .mapToLong(n -> n * n)
    .sum();
long tiempoParalelo = System.currentTimeMillis() - inicio;
```



En este ejemplo con 10 millones de números, el procesamiento paralelo es aproximadamente **4 veces más rápido** que el secuencial en un procesador de 8 núcleos. La mejora varía según el hardware y la naturaleza de la operación.

Conclusiones y Cierre de la Serie

- Elegir Sabiamente

Usa Streams paralelos cuando tengas **grandes cantidades de datos** y operaciones independientes y computacionalmente intensivas.

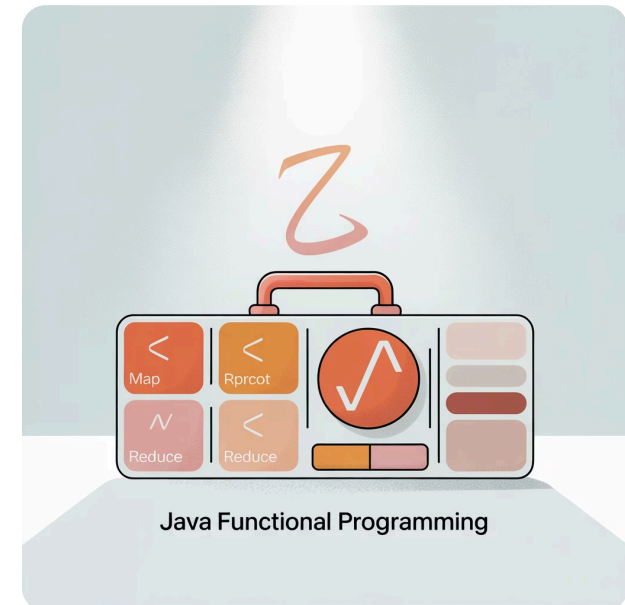
- Medir Siempre

No asumas que paralelo = más rápido. Realiza pruebas de rendimiento en tu entorno específico para confirmar los beneficios.

- Cuidado con los Efectos Secundarios

Evita modificar estado compartido en operaciones paralelas. Prefiere operaciones sin estado y colectores diseñados para paralelismo.

"Ahora tenéis todas las herramientas para escribir código más declarativo, expresivo y eficiente en Java."



Con esto completamos nuestro recorrido por la **Programación Funcional en Java**: lambdas, Streams, operaciones intermedias y terminales, Collectors, Optional y paralelismo. ¡Enhorabuena por llegar hasta aquí!